

Guía Didáctica del Patrón Repository

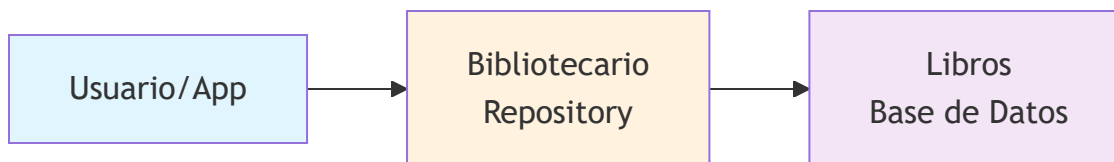
Índice

1. ¿Qué es el Patrón Repository?
2. ¿Por qué usarlo?
3. Problema que resuelve
4. Estructura del Patrón
5. Ejemplo Práctico paso a paso
6. Diagramas Mermaid
7. Ejercicios Propuestos

1. ¿Qué es el Patrón Repository?

El **Patrón Repository** es un patrón de diseño que actúa como un **mediador entre la lógica de negocio y la capa de acceso a datos**.

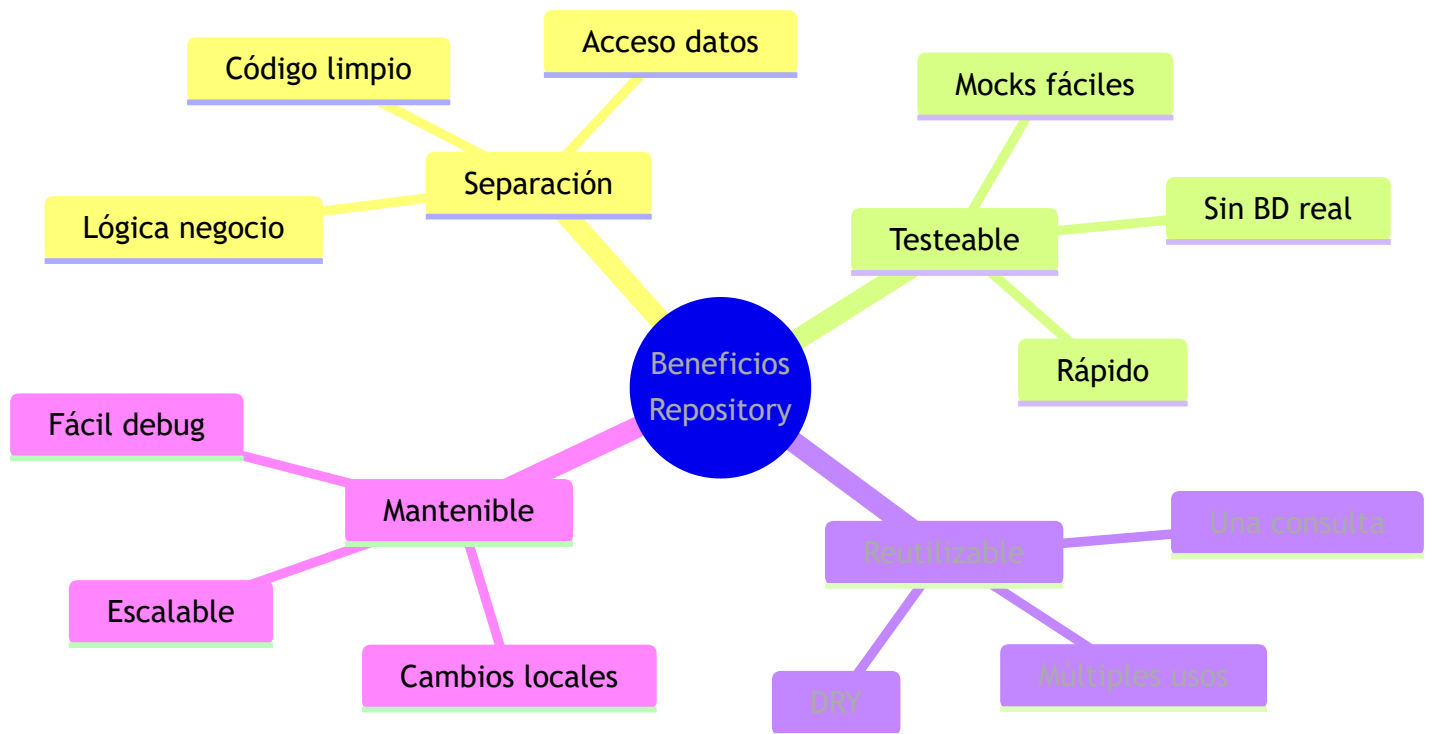
Analogía del bibliotecario



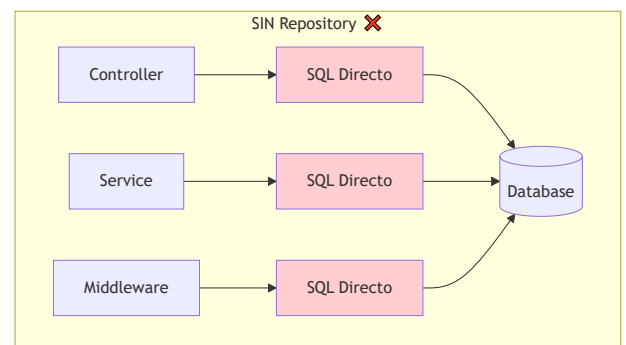
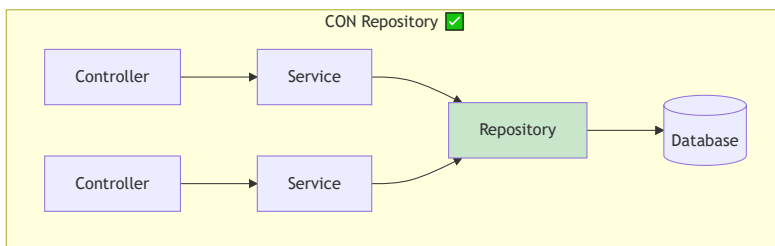
Explicación: Así como no necesitas saber dónde están exactamente los libros, tu aplicación no necesita saber cómo se guardan los datos. El Repository actúa como el bibliotecario.

2. ¿Por qué usarlo?

Beneficios principales

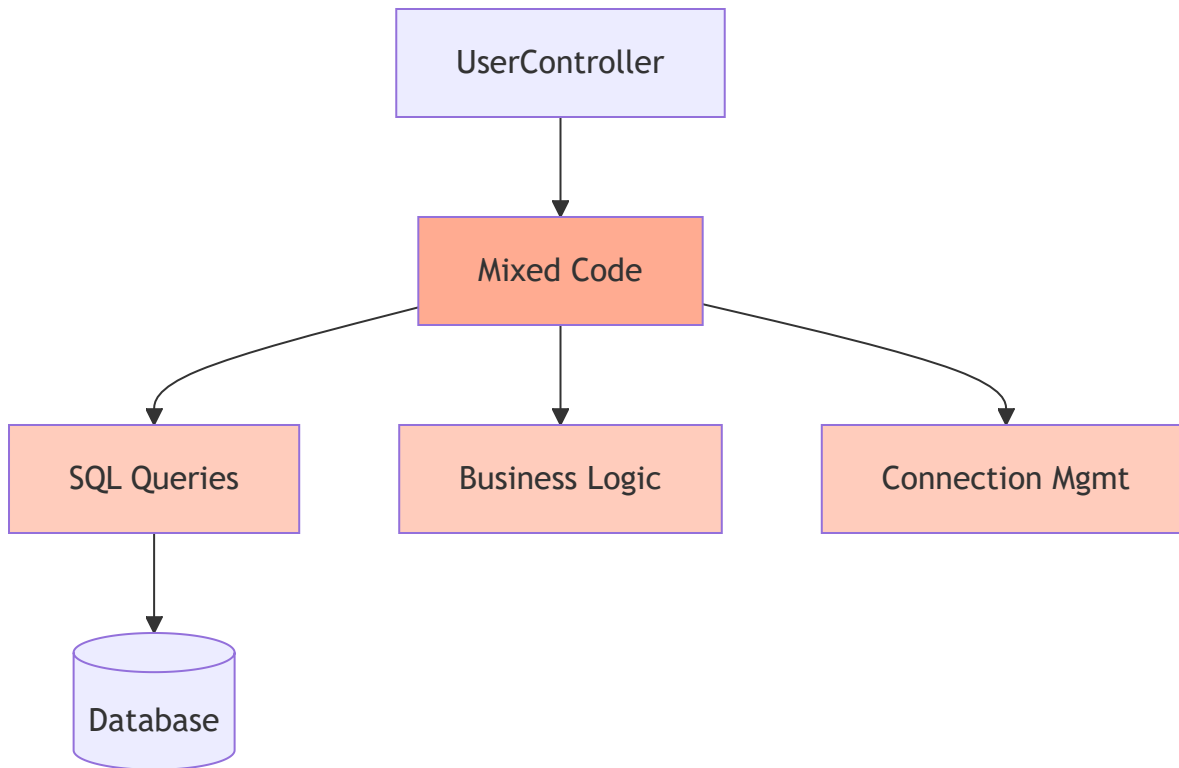


Comparativa visual



3. Problema que resuelve

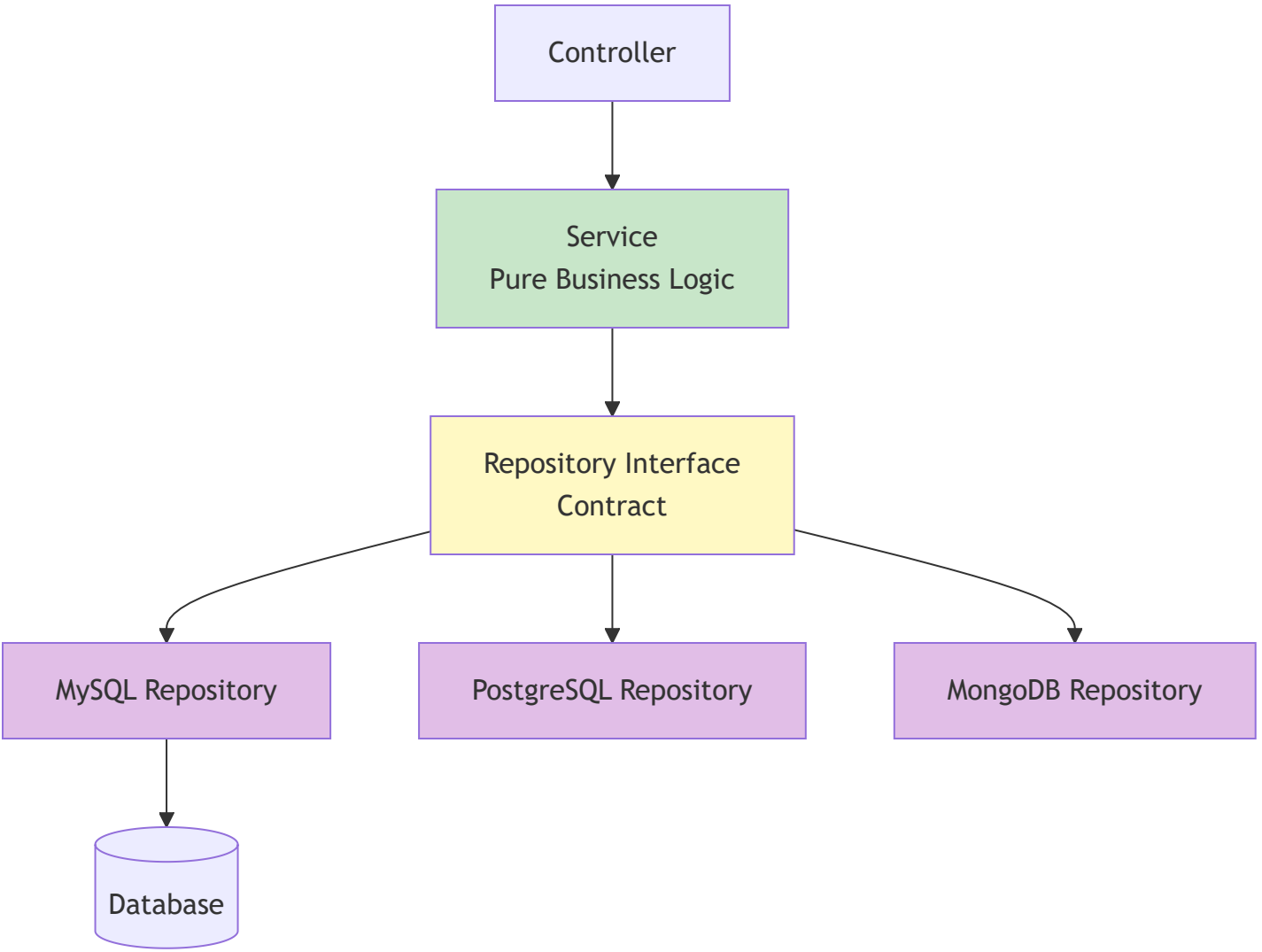
Código sin Repository (problemático)



Problemas:

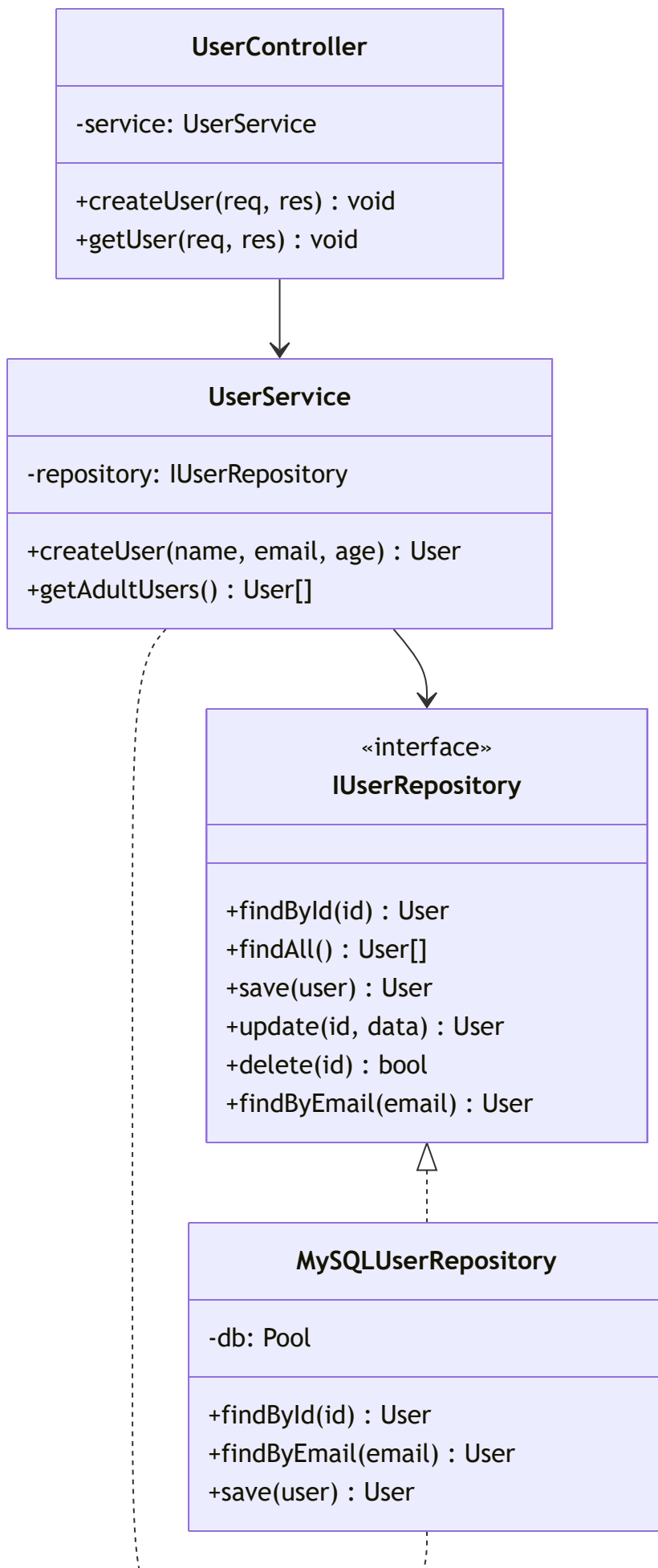
- ❌ Código duplicado
- ❌ SQL esparcido por toda la app
- ❌ Dependencia directa de la BD
- ❌ Tests complicados

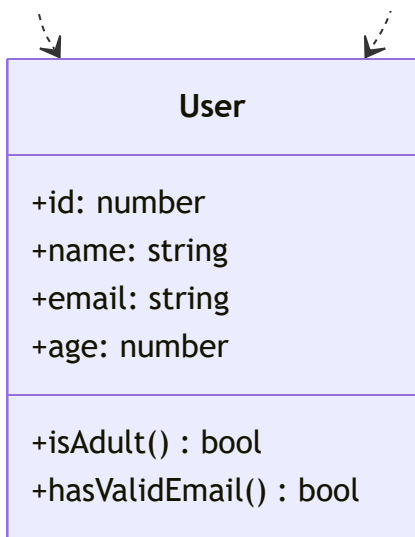
Código con Repository (solución)



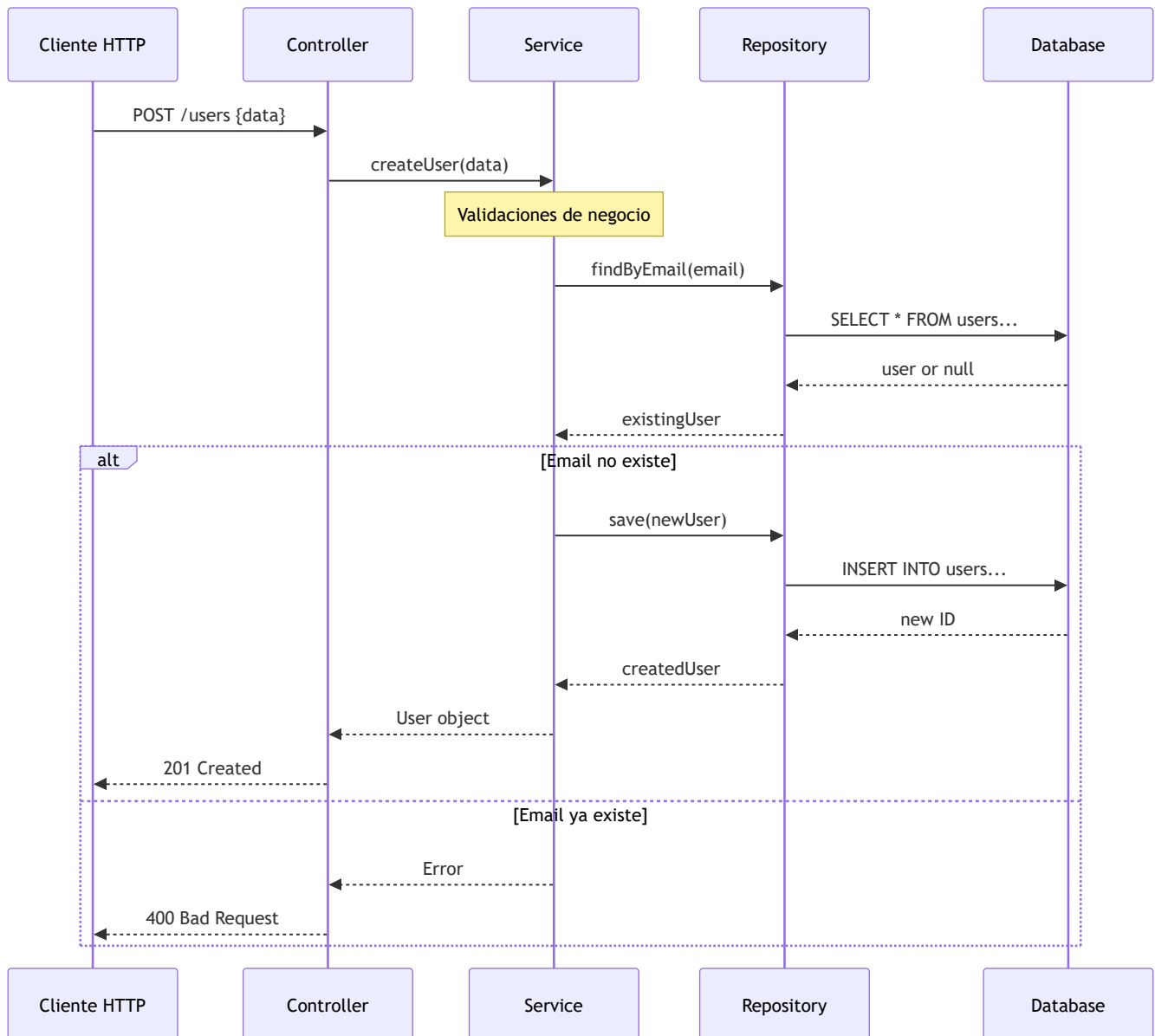
4. Estructura del Patrón

Diagrama de clases UML



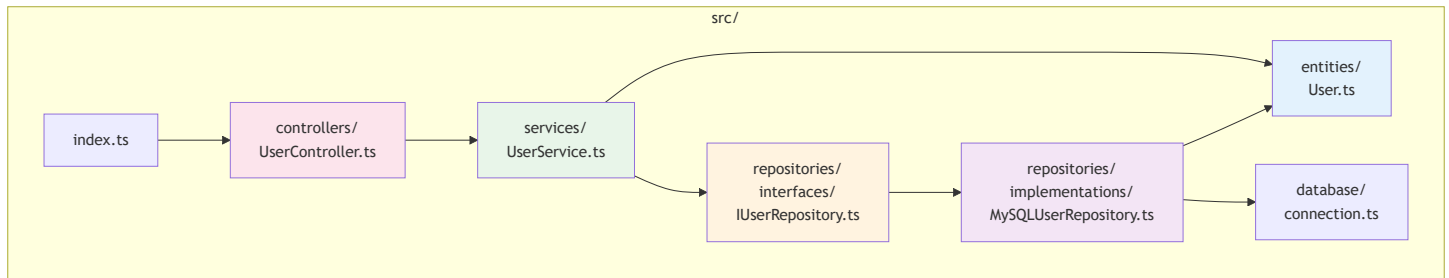


Flujo de datos completo



5. Ejemplo Práctico paso a paso

Estructura de carpetas



Código paso a paso

Paso 1: Entidad

```
// src/entities/User.ts
export class User {
  constructor(
    public id: number | null,
    public name: string,
    public email: string,
    public age: number,
    public createdAt: Date = new Date()
  ) {}

  isAdult(): boolean {
    return this.age >= 18;
  }
}
```

Paso 2: Interfaz del Repository

```
// src/repositories/interfaces/IUserRepository.ts
export interface IUserRepository {
  findById(id: number): Promise<User | null>;
  findAll(): Promise<User[]>;
  save(user: User): Promise<User>;
  update(id: number, user: Partial<User>): Promise<User>;
  delete(id: number): Promise<boolean>;
  findByEmail(email: string): Promise<User | null>;
}
```

Paso 3: Implementación con MySQL

```
// src/repositories/implementations/MySQLUserRepository.ts
import { Pool, RowDataPacket } from 'mysql2/promise';
import { DatabaseConnection } from '../../database/connection';
import { IUserRepository } from '../../interfaces/IUserRepository';
import { User } from '../../entities/User';

export class MySQLUserRepository implements IUserRepository {
  private db: Pool;

  constructor() {
    this.db = DatabaseConnection.getConnection();
  }

  async findById(id: number): Promise<User | null> {
    const [rows] = await this.db.execute<RowDataPacket[]>(
      'SELECT * FROM users WHERE id = ?',
      [id]
    );

    if (rows.length === 0) return null;

    const row = rows[0];
    return new User(
      row.id, row.name, row.email, row.age, new Date(row.created_at)
    );
  }

  async save(user: User): Promise<User> {
    const [result] = await this.db.execute(
      'INSERT INTO users (name, email, age, created_at) VALUES (?, ?, ?, ?)',
      [user.name, user.email, user.age, user.createdAt]
    );

    // @ts-ignore
    user.id = result.insertId;
    return user;
  }

  // ... resto de métodos
}
```

Paso 4: Servicio (Lógica de Negocio)

```
// src/services/UserService.ts
export class UserService {
  constructor(private userRepository: IUserRepository) {}

  async createUser(name: string, email: string, age: number): Promise<User> {
    // Validaciones de negocio
    if (age < 0 || age > 150) {
      throw new Error('Invalid age');
    }

    const existingUser = await this.userRepository.findByEmail(email);
    if (existingUser) {
      throw new Error('Email already exists');
    }

    const user = new User(null, name, email, age);

    if (!user.isValidEmail()) {
      throw new Error('Invalid email format');
    }

    return await this.userRepository.save(user);
  }

  async getAdultUsers(): Promise<User[]> {
    const users = await this.userRepository.findAll();
    return users.filter(user => user.isAdult());
  }
}
```

Paso 5: Controlador

```
// src/controllers/UserController.ts
export class UserController {
  constructor(private userService: UserService) {}

  async createUser(req: Request, res: Response): Promise<void> {
    try {
      const { name, email, age } = req.body;
      const user = await this.userService.createUser(name, email, age);
      res.status(201).json({ success: true, data: user });
    } catch (error) {
      res.status(400).json({ success: false, error: error.message });
    }
  }
}
```

Paso 6: Inyección de dependencias

```
// src/index.ts
const userRepository = new MySQLUserRepository();
const userService = new UserService(userRepository);
const userController = new UserController(userService);

app.post('/users', (req, res) => userController.createUser(req, res));
```

6. Diagramas Mermaid

Diagrama de componentes

Diagrama de Componentes - Sistema con Repository Pattern

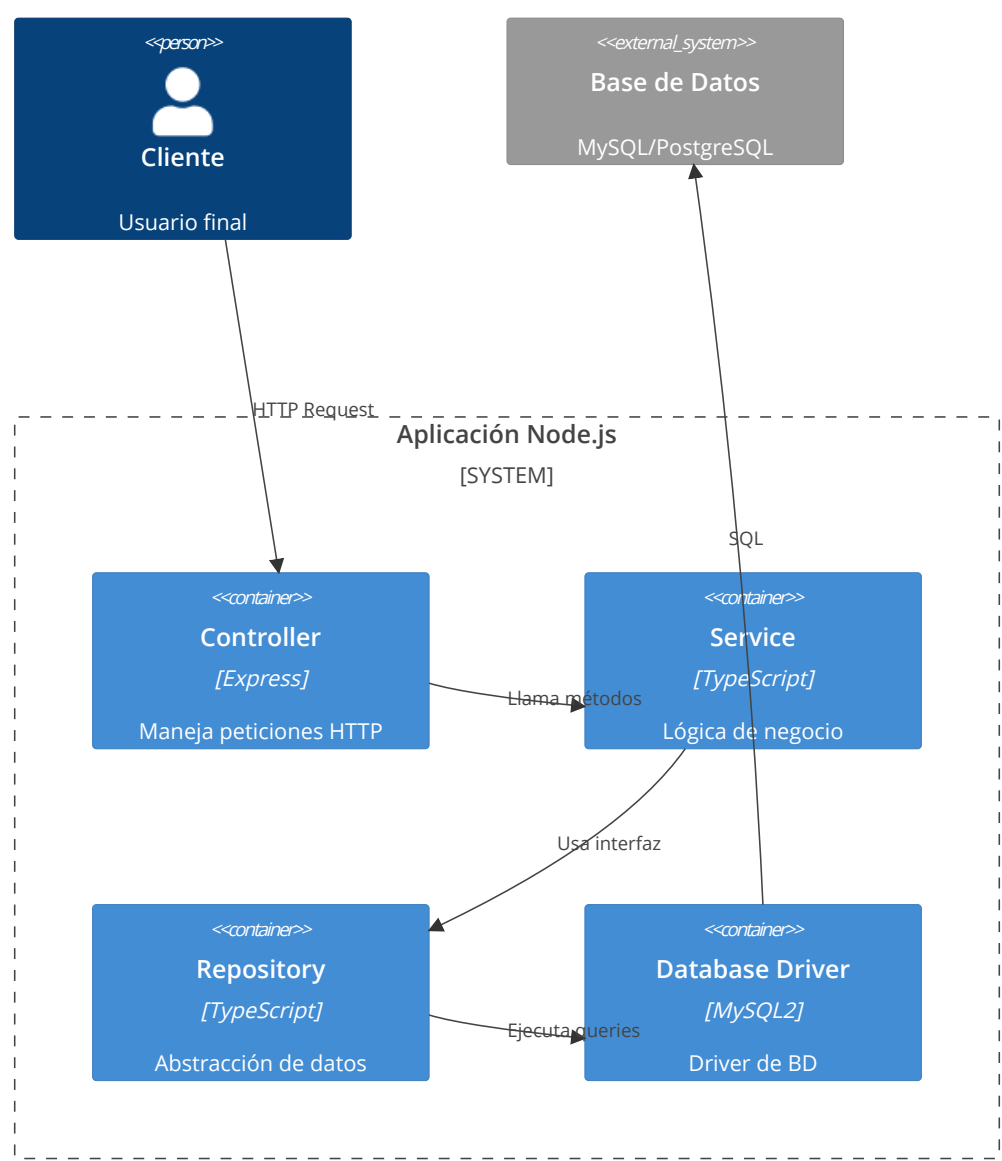
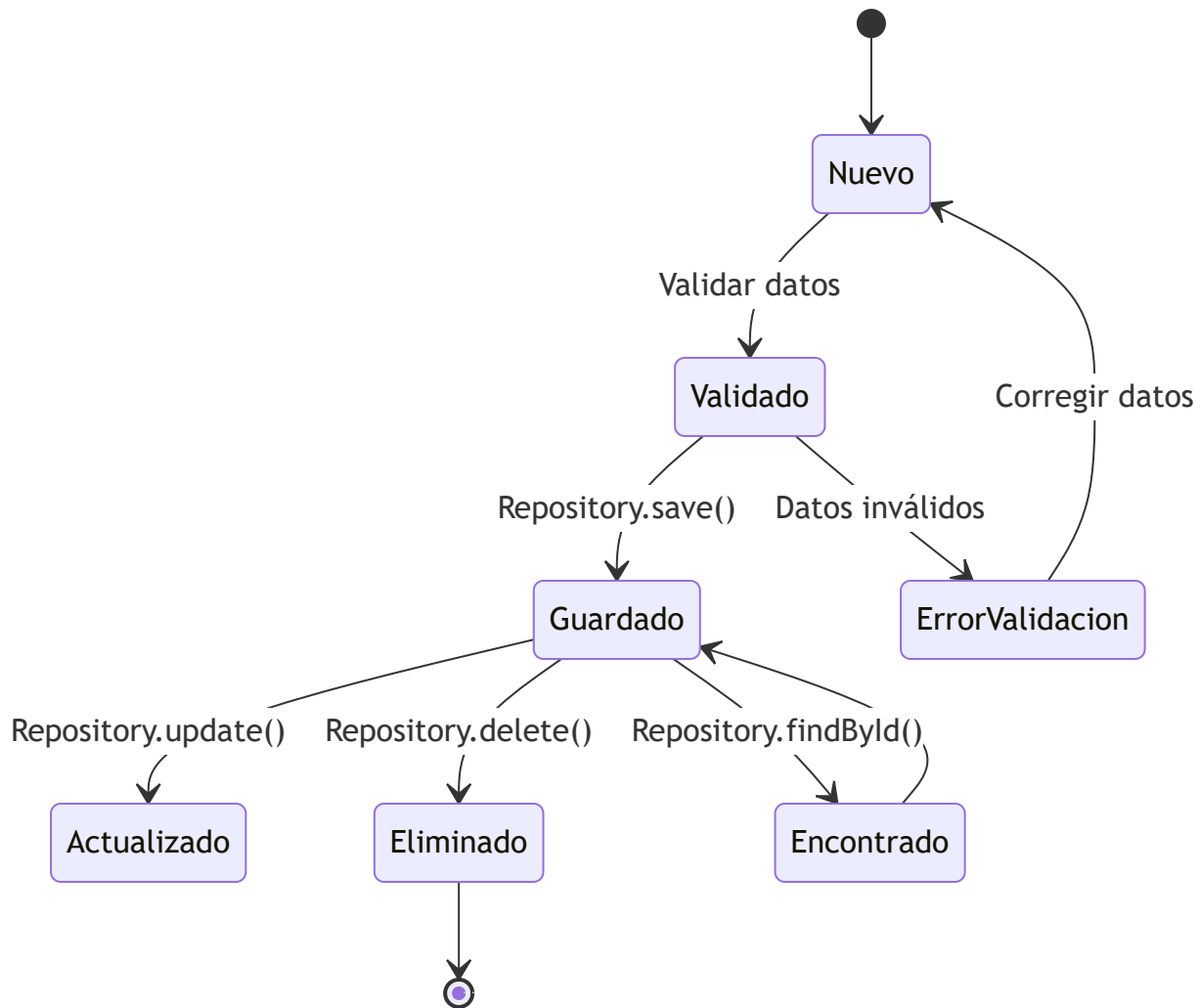
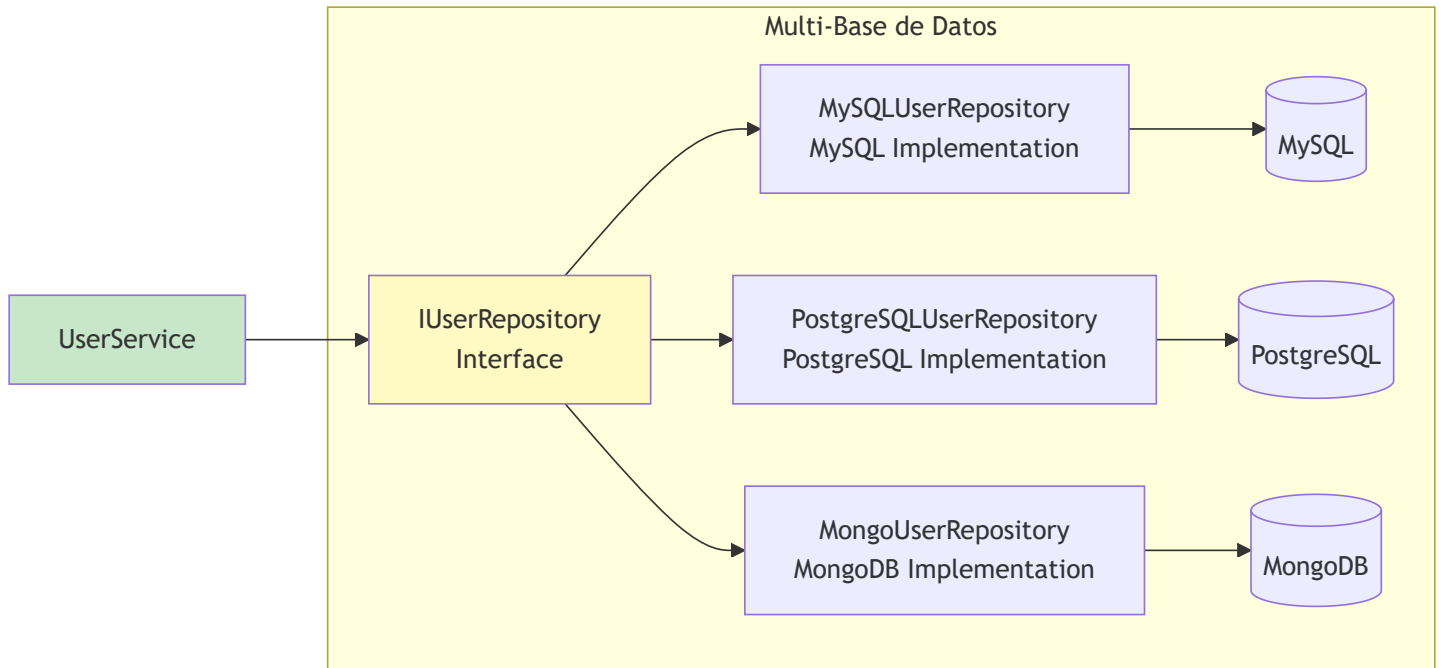


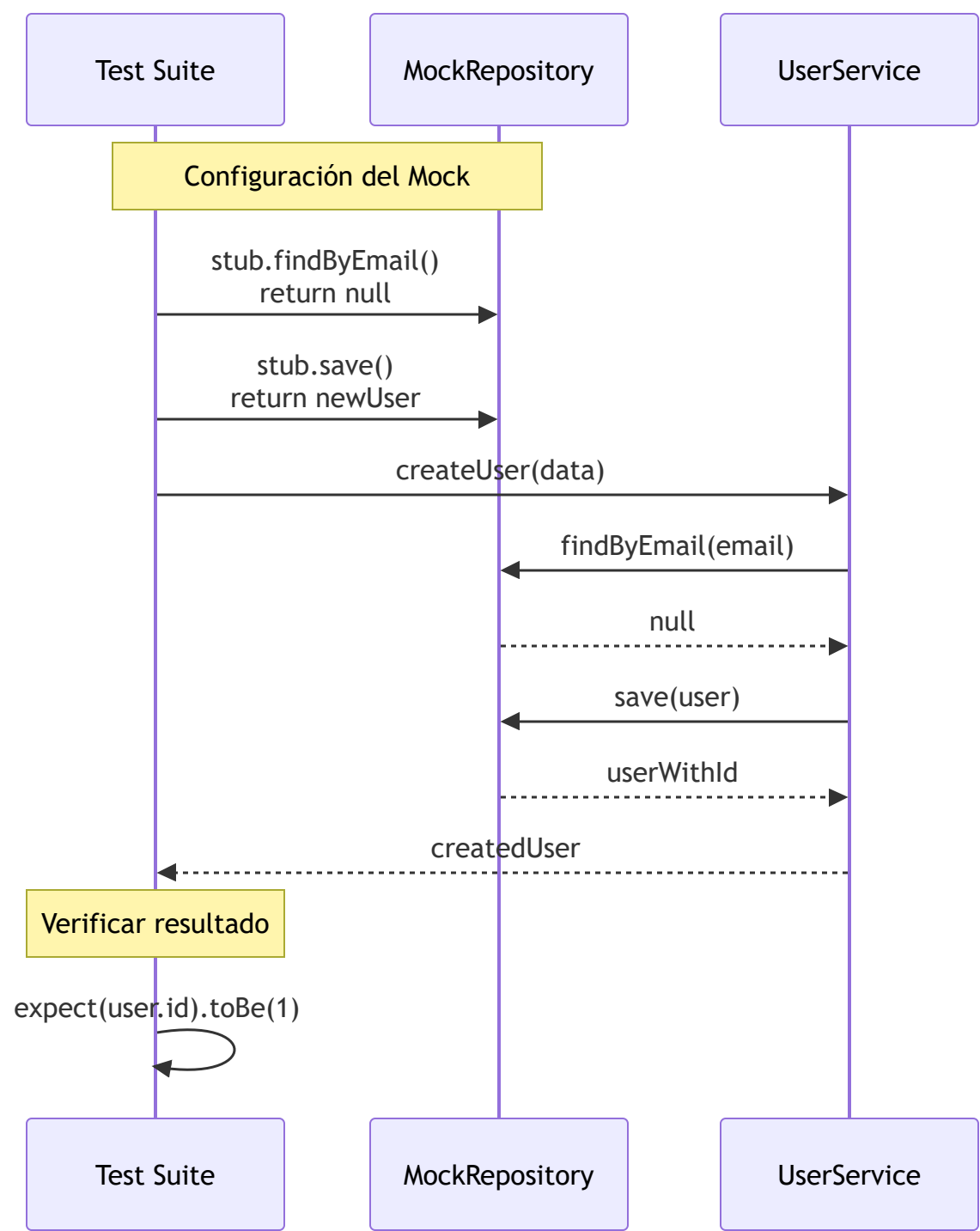
Diagrama de estado de una entidad



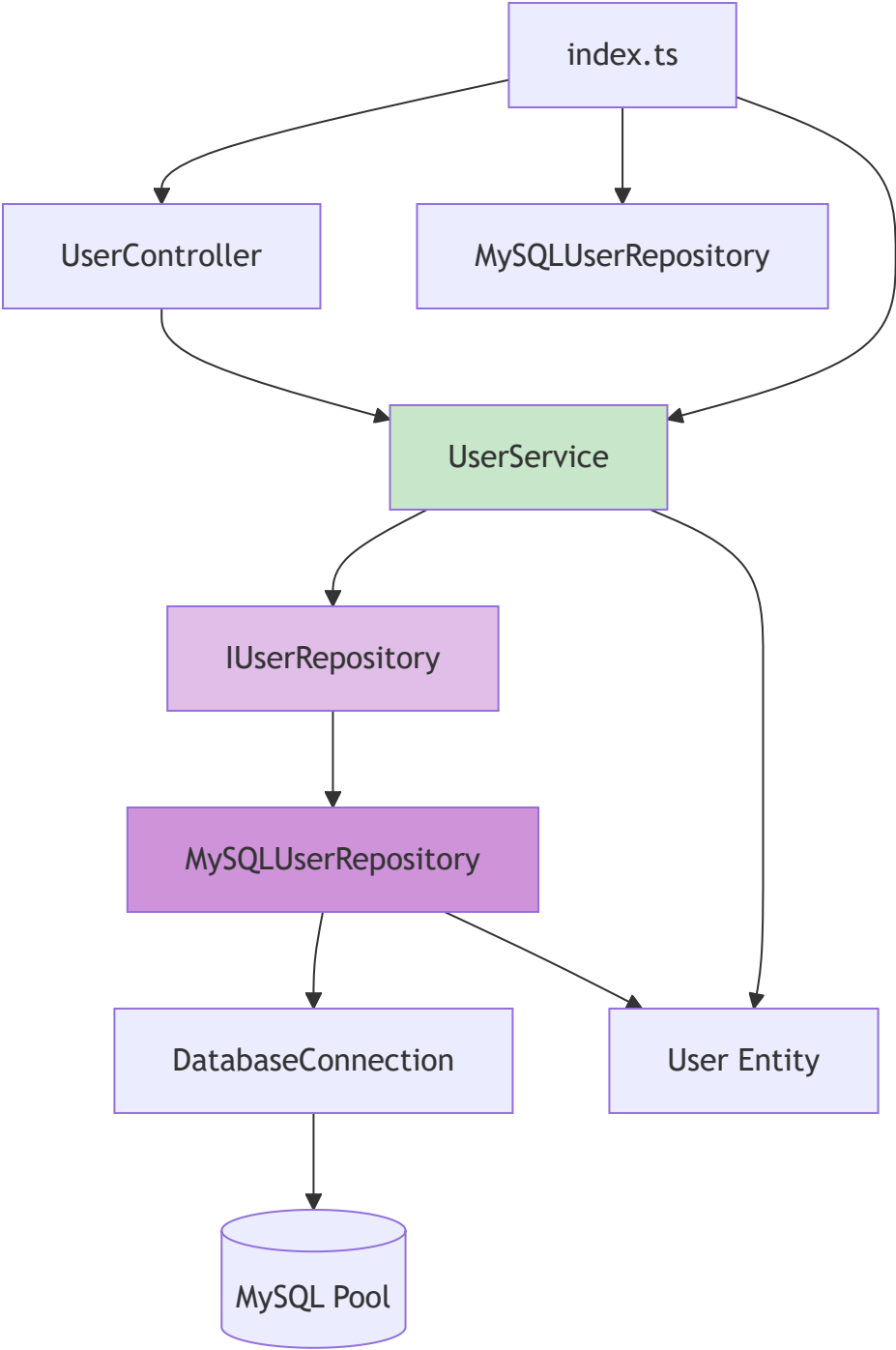
Comparativa de implementaciones



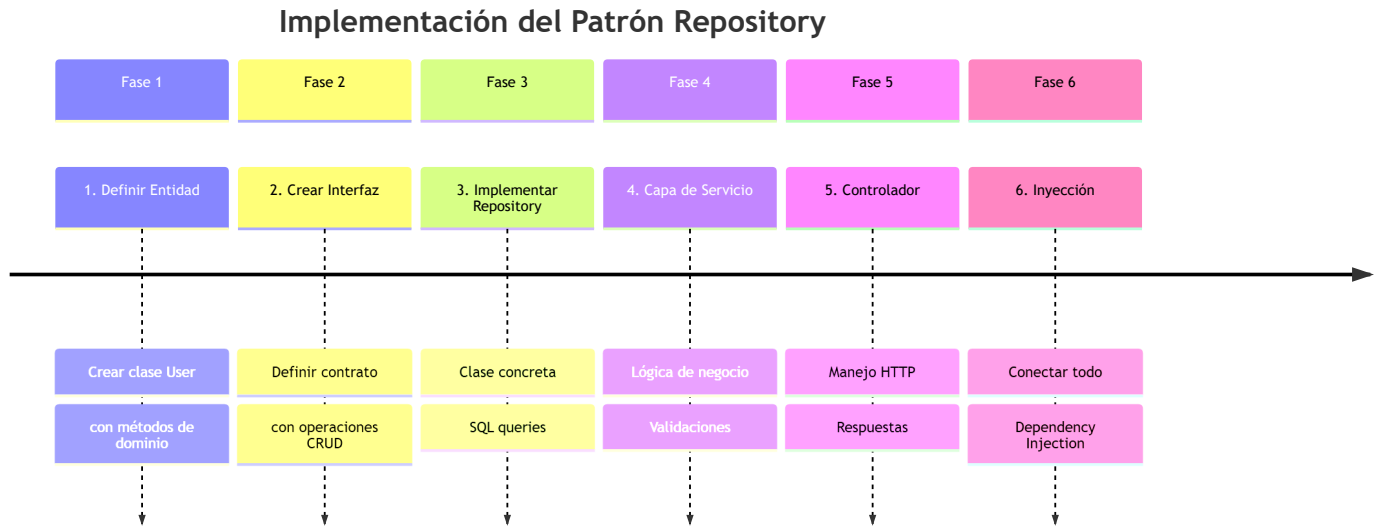
Flujo de testing con mocks



Árbol de dependencias



Evolución del proyecto



Mapa mental de decisiones

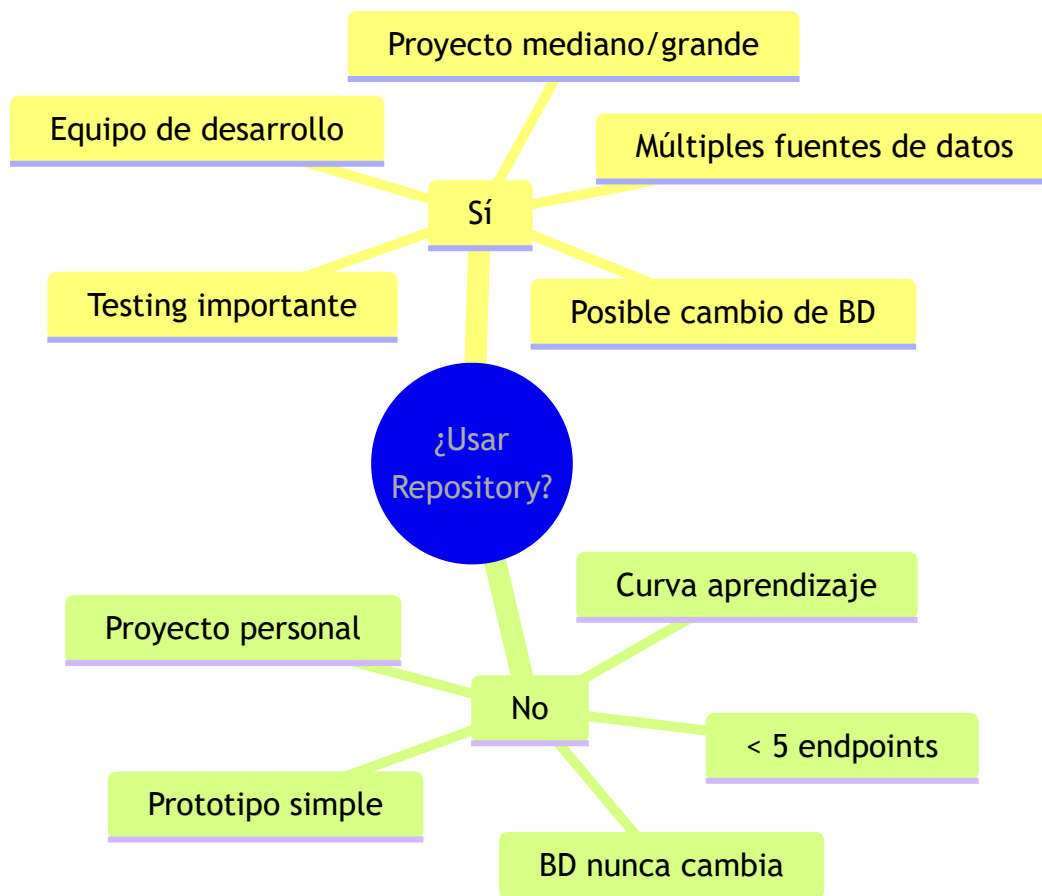


Diagrama de paquetes

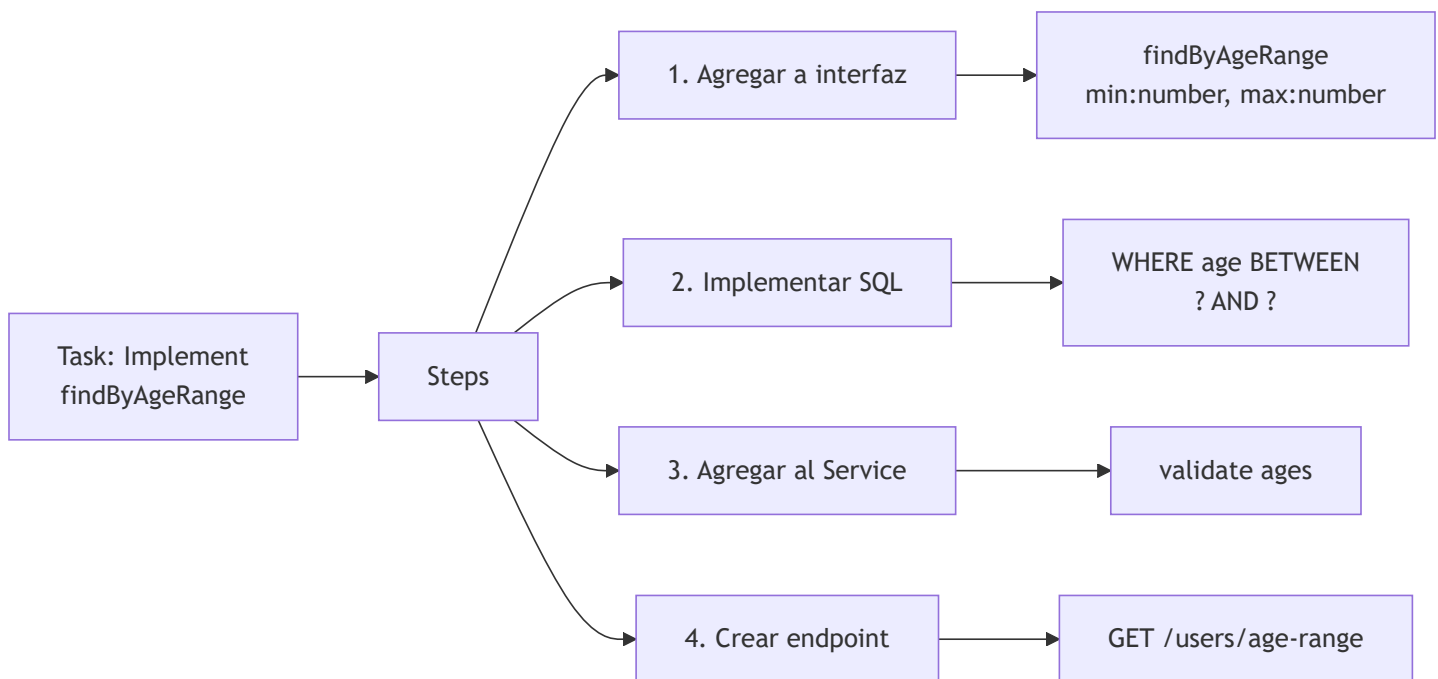


Syntax error in text
mermaid version 11.15.0

7. Ejercicios Propuestos

Ejercicio 1: Implementar nuevo método

Objetivo: Agregar búsqueda por rango de edad.

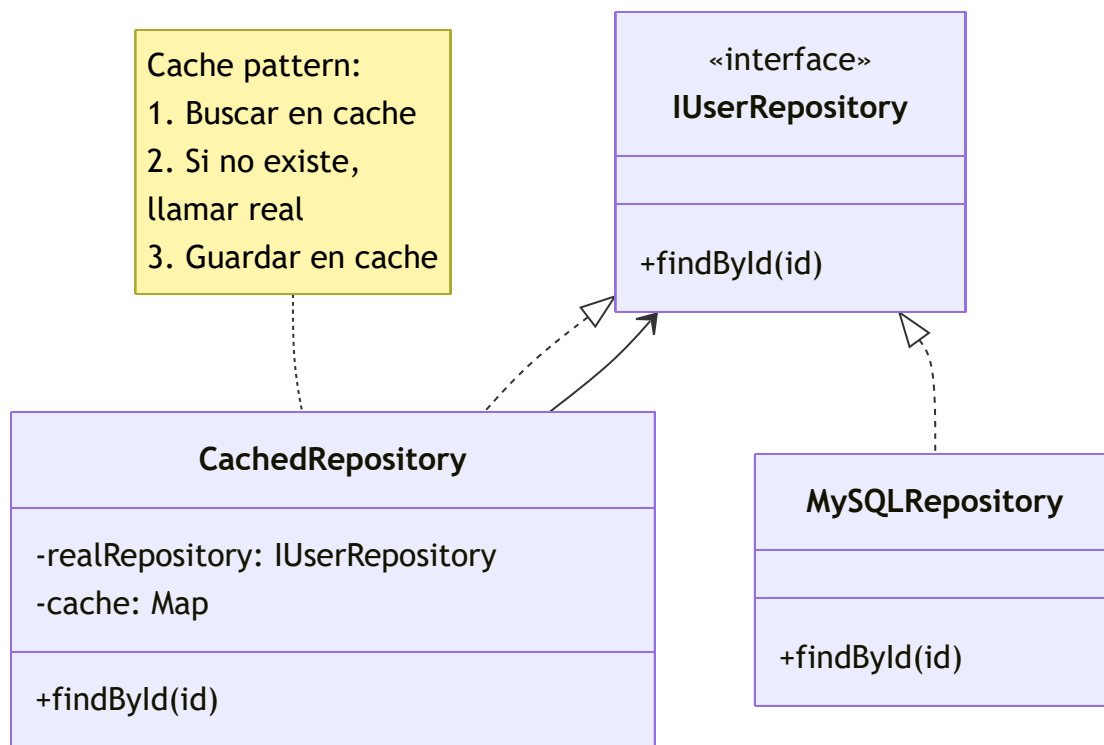


Ejercicio 2: Cambiar a MongoDB

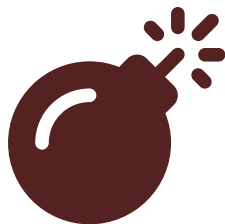


Syntax error in text
mermaid version 11.15.0

Ejercicio 3: Añadir caché con Decorator

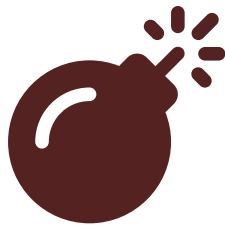


Ejercicio 4: Testing completo



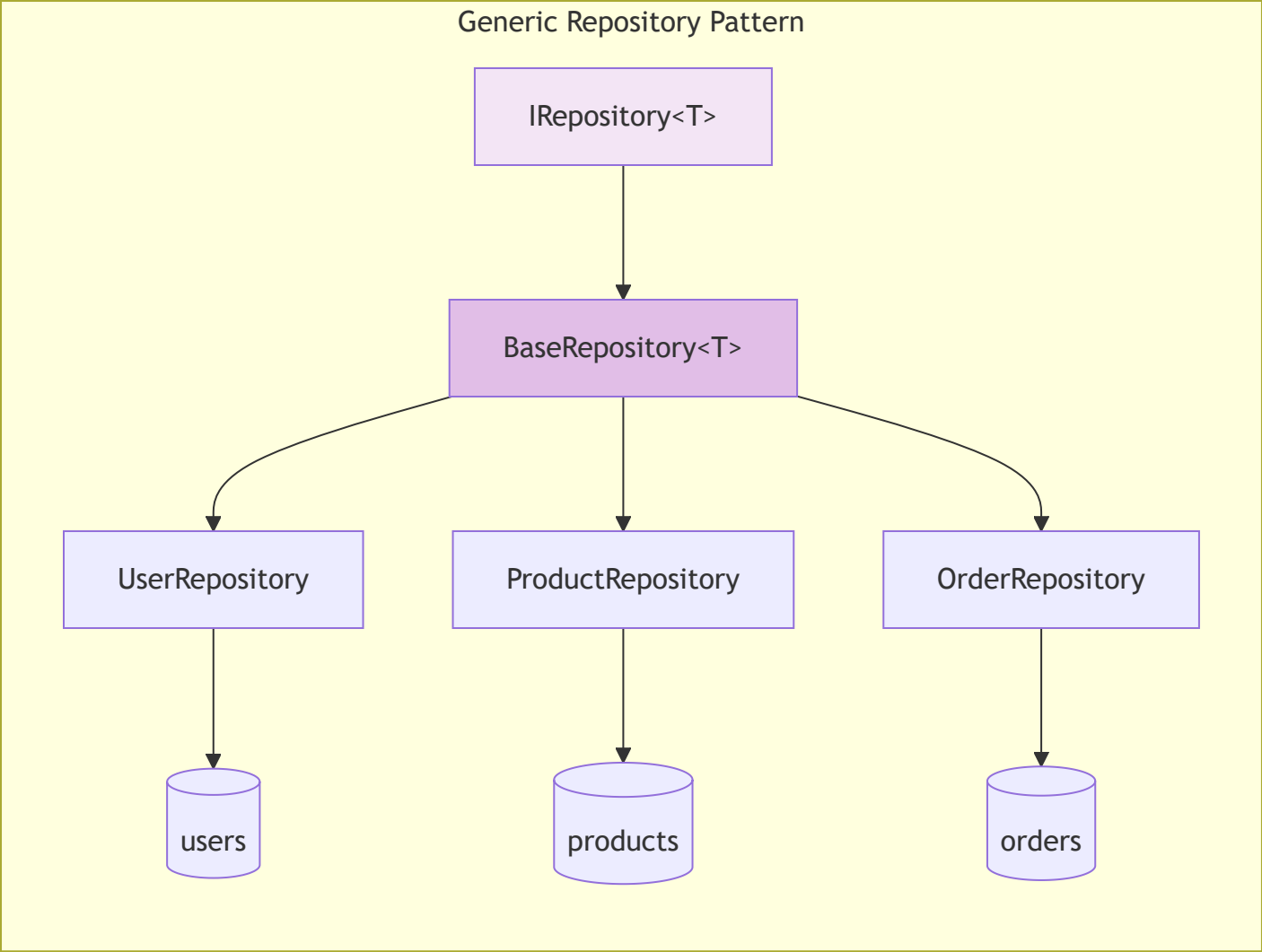
Syntax error in text
mermaid version 11.15.0

Ejercicio 5: Repository Genérico

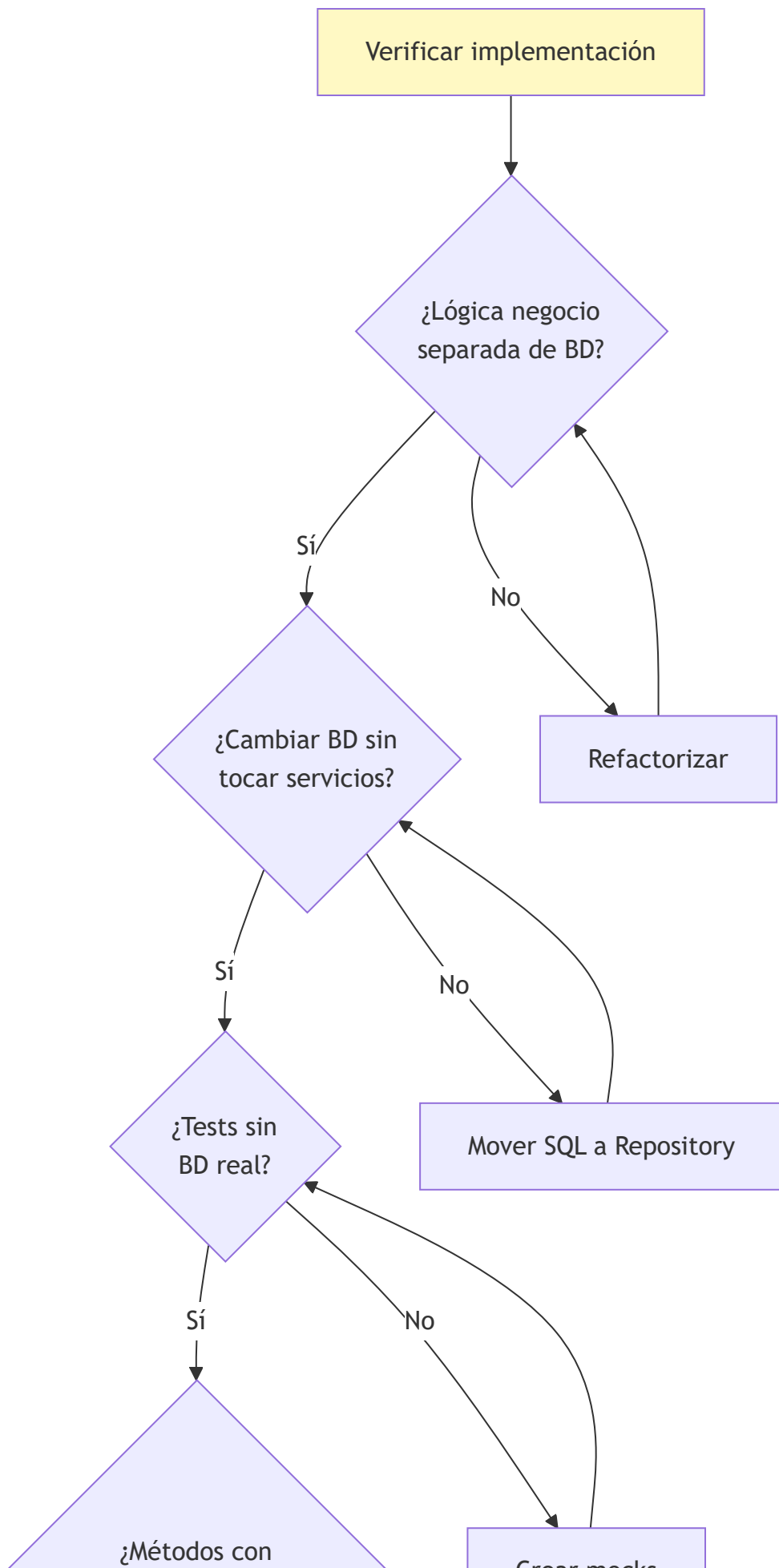


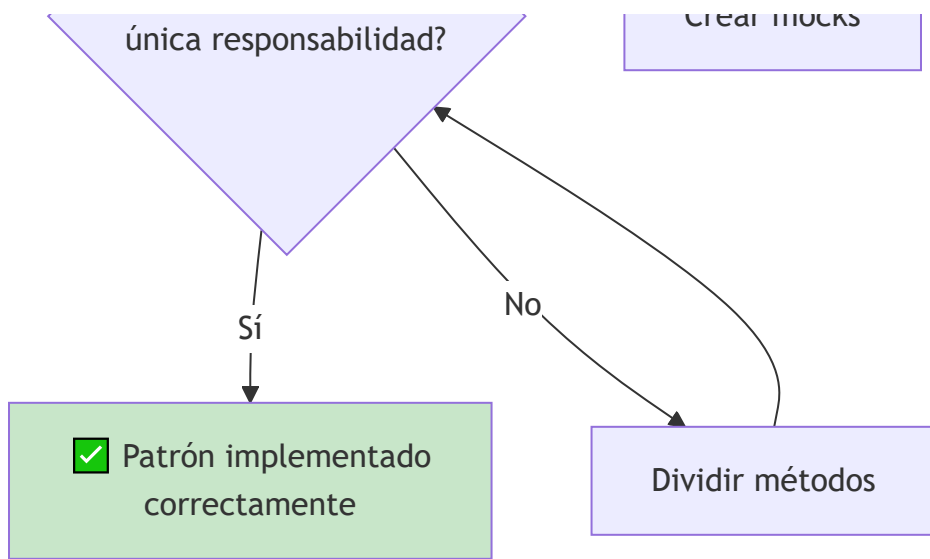
Syntax error in text
mermaid version 11.15.0

Diagrama de solución del desafío final

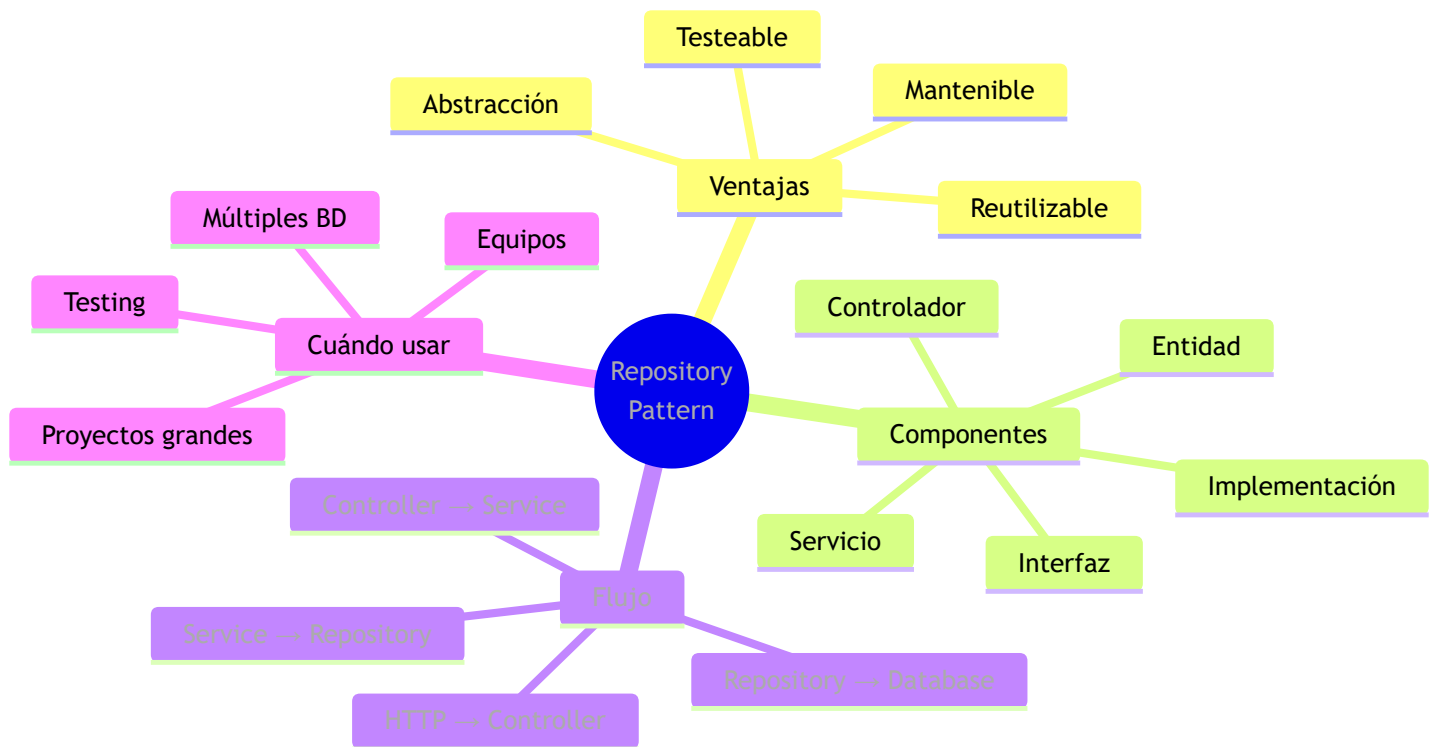


Checklist de verificación





Resumen visual



Recursos adicionales

Enlaces útiles

Documentación
TypeScript

MySQL2
NPM

Repository Pattern
Martin Fowler

Ejemplos
GitHub